

Day 6: Object-Oriented Programming (OOP) in Python

Welcome to Day 6! Today we explore **Object-Oriented Programming (OOP)**, a programming paradigm that structures code using classes and objects. We will cover:

1. **Core Concepts:** Classes, Objects, Instantiation, and the `self` parameter.
 2. **Attributes & Scopes:** Instance vs. Class variables, and references.
 3. **OOP Decorators:** `@classmethod`, `@staticmethod`, and `@property`.
 4. **Inheritance & MRO:** Single/Multiple inheritance, and cooperative super calls.
 5. **Polymorphism:** Method overriding and overloading behavior.
 6. **Encapsulation:** Private/Protected naming conventions and name mangling.
 7. **Special Dunder Methods:** Representation (`__str__`, `__repr__`), Operator Overloading (`__add__`, `__eq__`), and Custom Iterators (`__iter__`, `__next__`).
-

Part 1: Core OOP Concepts

1. Classes, Objects, and Instantiation

- **Class:** A user-defined blueprint or template for creating objects.
- **Object:** An instance of a class containing real values and executable behaviors.
- **Instantiation:** The process of allocating memory and initializing a new object.

```
class Student:
    # Constructor/Initializer method
    def __init__(self, name, age):
        self.name = name    # Instance attribute
        self.age = age      # Instance attribute

    # Instance method
    def display_details(self):
        return f"Student: {self.name}, Age: {self.age}"

# Instantiation
student_1 = Student("Arham", 21)
print(student_1.display_details())  # Output: Student: Arham, Age: 21
```

2. The `self` Parameter

In Python, `self` represents the specific instance of the class that is currently invoking the method.

- You must include `self` as the first parameter in all instance methods.
- When you call the method as `obj.method()`, Python automatically passes the object reference as the first argument (`self`).

3. Instance Variables vs. Class Variables

- **Instance Variables:** Defined inside methods (usually `__init__`) prefixed with `self.`. They belong to a specific object instance.
- **Class Variables:** Defined directly inside the class body but outside any methods. They are shared across all instances of the class.

```
class CDACStudent:
    course = "PGCP-AI" # Class Variable (shared by all)

    def __init__(self, name):
        self.name = name # Instance Variable (unique to each)

s1 = CDACStudent("Arham")
s2 = CDACStudent("Lisa")

print(s1.name, "| Course:", s1.course) # Arham | Course: PGCP-AI
print(s2.name, "| Course:", s2.course) # Lisa | Course: PGCP-AI
```

Part 2: OOP Decorators

Python provides built-in decorators to modify class method behavior.

1. Class Methods (@classmethod)

- Receives the class (`cls`) as the first parameter instead of `self`.
- Can modify class state that applies to all instances.
- Often used to define "factory methods" (alternative constructors).

```
class DateConverter:
    def __init__(self, year, month, day):
        self.year, self.month, self.day = year, month, day

    @classmethod
    def from_string(cls, date_str):
        # Parses "YYYY-MM-DD" and creates a new object
        parts = list(map(int, date_str.split("-")))
        return cls(parts[0], parts[1], parts[2])

# Use the factory classmethod to instantiate
date_obj = DateConverter.from_string("2026-08-28")
print(date_obj.year) # Output: 2026
```

2. Static Methods (@staticmethod)

- Does not receive `self` or `cls` parameters.
- Behaves exactly like a standard function, but resides inside the class namespace.
- Used for helper or utility functions that don't need to access or modify class/instance state.

```
class MathUtility:
    @staticmethod
    def is_even(num):
        return num % 2 == 0

print(MathUtility.is_even(10)) # Output: True
```

3. Properties (@property)

- Converts a method call into a read-only attribute getter.
- Combined with `.setter` and `.deleter` decorators, properties allow you to enforce validations on attribute updates.

```
class Account:
    def __init__(self, balance):
        self.__balance = balance # Private attribute

    @property
    def balance(self):
        """Getter property."""
        return self.__balance

    @balance.setter
    def balance(self, new_val):
        """Setter property with validation."""
        if new_val < 0:
            raise ValueError("Balance cannot be negative.")
        self.__balance = new_val

acc = Account(100.0)
print(acc.balance) # Output: 100.0 (called without parenthesis)
acc.balance = 150.0 # Invokes the setter
# acc.balance = -50.0 # Raises ValueError
```

Part 3: Inheritance & Method Resolution Order (MRO)

1. Single Inheritance

A child class inherits attributes and methods from a single parent class. Use `super()` to invoke parent methods.

```
class Person:
    def __init__(self, name):
        self.name = name

class Employee(Person):
```

```
def __init__(self, name, emp_id):  
    super().__init__(name) # Initialize parent class attributes  
    self.emp_id = emp_id
```

2. Multiple Inheritance & MRO

A class can inherit from multiple parent classes.

- **Method Resolution Order (MRO):** The order in which Python searches for a method or attribute in a class hierarchy.
- You can inspect this order using the `__mro__` attribute or `.mro()` method.

```
class A:  
    def process(self):  
        print("Process A")  
  
class B(A):  
    def process(self):  
        print("Process B")  
        super().process()  
  
class C(A):  
    def process(self):  
        print("Process C")  
        super().process()  
  
class D(B, C):  
    def process(self):  
        print("Process D")  
        super().process()  
  
d = D()  
d.process()  
# Output order shows the cooperative MRO resolution:  
# Process D -> Process B -> Process C -> Process A  
  
print(D.__mro__)  
# Output: (<class 'D'>, <class 'B'>, <class 'C'>, <class 'A'>, <class  
'object'>)
```

Part 4: Polymorphism

Polymorphism allows different classes to define methods with the same name.

1. Method Overriding

A subclass provides a specific implementation of a method that is already defined by its parent class.

```
class Animal:
    def make_sound(self):
        return "Generic Sound"

class Dog(Animal):
    def make_sound(self):
        return "Woof"

class Cat(Animal):
    def make_sound(self):
        return "Meow"

animals = [Dog(), Cat()]
for animal in animals:
    print(animal.make_sound()) # Woof, then Meow
```

2. Method Overloading (in Python)

Unlike Java or C++, Python does not support standard method overloading (defining multiple methods with the same name but different signatures). In Python, the last method definition overrides all previous ones.

To implement overloading behavior, use default parameters or variable arguments (***args**):

```
class Calculator:
    def add(self, a, b, c=None):
        if c is not None:
            return a + b + c
        return a + b

calc = Calculator()
print(calc.add(2, 3)) # Output: 5
print(calc.add(2, 3, 5)) # Output: 10
```

Part 5: Encapsulation & Data Hiding

Encapsulation restricts direct access to some of an object's components.

- **Public:** Accessible from anywhere (default). E.g., **self.name**.
- **Protected:** A convention indicating the variable should not be accessed outside the class. Prefixed with a single underscore. E.g., **self._name**.
- **Private:** Restricts direct access. Prefixed with double underscores. E.g., **self.__name**.
 - **Name Mangling:** Python replaces double-underscore variable names under the hood with **_ClassName__variable_name** to prevent external access.

```
class SecureDevice:
    def __init__(self, key):
        self.__secret_key = key # Private attribute

device = SecureDevice("12345")

# Trying to access directly raises AttributeError
try:
    print(device.__secret_key)
except AttributeError:
    print("Cannot access private variable __secret_key")

# Accessing via name mangling (Discouraged, but possible)
print("Mangling access:", device._SecureDevice__secret_key) # Output:
12345
```

Part 6: Special Dunder Methods

Special methods are prefixed and suffixed with double underscores (`__`). They allow objects to integrate with Python built-in behaviors.

1. String Representation: `__str__` vs. `__repr__`

- `__str__`: Returns a user-friendly string representation of the object (called by `print()` or `str()`).
- `__repr__`: Returns an unambiguous, developer-friendly string representation (called by `repr()` or in interactive shells).

```
class Coordinates:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __str__(self):
        return f"({self.x}, {self.y})"

    def __repr__(self):
        return f"Coordinates(x={self.x}, y={self.y})"

pt = Coordinates(3, 4)
print(str(pt)) # Output: (3, 4)
print(repr(pt)) # Output: Coordinates(x=3, y=4)
```

2. Operator Overloading

You can define custom behavior for mathematical and comparison operators:

```
class Money:
    def __init__(self, amount):
        self.amount = amount

    def __add__(self, other):
        """Overloads the + operator."""
        if not isinstance(other, Money):
            raise TypeError("Can only add Money objects.")
        return Money(self.amount + other.amount)

    def __eq__(self, other):
        """Overloads the == operator."""
        if not isinstance(other, Money):
            return False
        return self.amount == other.amount

m1 = Money(10)
m2 = Money(20)
m3 = m1 + m2
print(m3.amount) # Output: 30
print(m1 == Money(10)) # Output: True
```

3. Custom Iterators (`__iter__` and `__next__`)

An object can be made iterable by implementing the iterator protocol:

```
class Countdown:
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        val = self.current
        self.current -= 1
        return val

for num in Countdown(3):
    print(num)
# Output:
# 3
# 2
# 1
```